Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main()
{
    int n, i;
    int nums[50000];
    int candidate = 0;
    int count = 0;

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    for (i = 0; i < n; i++)
    {
        if (count == 0)
        {
            candidate = nums[i];
            count = 1;
        }
        else if (nums[i] == candidate)
        {
            count++;
        }
        else
        {
            count--;
        }
    }

    printf("%d", candidate);
```

```
35
36      return 0;
37  }
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main()
{
    int n, i, x;
    int a[100000];
    int low = 0, high, mid;
    int floor = -1;

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    scanf("%d", &x);

    high = n - 1;

    while (low <= high)
    {
        mid = low + (high - low) / 2;

        if (a[mid] == x)
        {
            floor = a[mid];
            break;
        }
        else if (a[mid] < x)
        {
            floor = a[mid];
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    printf("%d", floor);

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findPair(int a[], int left, int right, int x)
{
    if (left >= right)
        return 0;

    int sum = a[left] + a[right];

    if (sum == x)
    {
        printf("%d\n", a[left]);
        printf("%d\n", a[right]);
        return 1;
    }

    if (sum < x)
        return findPair(a, left + 1, right, x);
    else
        return findPair(a, left, right - 1, x);
}

int main()
{
    int n, i, x;
    int a[100000];

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    scanf("%d", &x);

    if (!findPair(a, 0, n - 1, x))
    {
        printf("No");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1;
    int j;

    for (j = low; j < high; j++)
    {
        if (a[j] < pivot)
        {
            i++;
            swap(&a[i], &a[j]);
        }
    }

    swap(&a[i + 1], &a[high]);

    return i + 1;
}

void quickSort(int a[], int low, int high)
{
    if (low < high)
    {
        int p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main()
{
    int n, i;
    int a[100000];

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course